

sql Relationship One to One explain all concept

SQL One-to-One Relationship (1:1) – Complete Concept

A One-to-One Relationship means:

One row in Table A is connected to only one row in Table B.

And

One row in Table B is connected to only one row in Table A.

Real Life Examples

Table A	Table B	Relationship
Person	Passport	One person has one passport
User	UserProfile	One user has one profile
Employee	ParkingSpace	One employee gets one parking space
Student	StudentIDCard	One student has one ID card

Example Diagram

```
USERS
+-----+
| id | name |
+-----+
| 1  | Aamir |
| 2  | Ali   |
+-----+
```

USER_PROFILE

```
+-----+-----+-----+
| id | user_id | bio   |
+-----+-----+-----+
| 1  | 1       | Coder |
| 2  | 2       | Designer |
+-----+-----+-----+
```

Here:

- User 1 → only one profile
- Profile 1 → belongs to only one user

This is **One-to-One**.

How SQL Creates One-to-One Relationship

We use:

1. Primary Key
2. Foreign Key
3. UNIQUE Constraint

Important Terms

1. Primary Key

Uniquely identifies each row.

SQL

```
id INT PRIMARY KEY
```

2. Foreign Key

Connects one table to another.

SQL

```
user_id INT,
FOREIGN KEY (user_id) REFERENCES users(id)
```

3. UNIQUE Constraint

Makes sure one value appears only once.

SQL

```
user_id INT UNIQUE
```

This is the MOST IMPORTANT part for One-to-One.

Method 1: Separate Primary Key + Unique Foreign Key

Create Parent Table

SQL

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  name VARCHAR(50)  
);
```

Create Child Table

SQL

```
CREATE TABLE user_profile (  
  id INT PRIMARY KEY,  
  user_id INT UNIQUE,  
  bio VARCHAR(100),  
  
  FOREIGN KEY (user_id)  
  REFERENCES users(id)  
);
```

Why UNIQUE is Important?

Without `UNIQUE`, many profiles could use same `user_id`.

That would become:

✗ One-to-Many

Example:

```
user_id = 1  
user_id = 1  
user_id = 1
```

So we add:

SQL

```
UNIQUE(user_id)
```

Now only one profile per user is allowed.

Insert Data

Insert Users

SQL

```
INSERT INTO users VALUES  
(1, 'Aamir'),  
(2, 'Ali');
```

Insert Profiles

SQL

```
INSERT INTO user_profile VALUES  
(1, 1, 'Web Developer'),  
(2, 2, 'Designer');
```

Invalid Insert Example

SQL

```
INSERT INTO user_profile VALUES  
(3, 1, 'Another Bio');
```

✗ ERROR

Because:

```
user_id = 1
```

already exists.

UNIQUE constraint prevents duplicate relationship.

Method 2: Shared Primary Key Relationship

This is a stronger One-to-One relationship.

Both tables share same primary key.

Example

Parent Table

SQL

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  name VARCHAR(50)  
);
```

Child Table

SQL

```
CREATE TABLE profiles (  
  id INT PRIMARY KEY,
```

```
bio VARCHAR(100),  
  
FOREIGN KEY (id)  
REFERENCES users(id)  
);
```

How It Works

```
users.id = profiles.id
```

Same ID used in both tables.

Example:

```
users.id = 1  
profiles.id = 1
```

This guarantees exactly one matching row.

Difference Between Both Methods

Method	Description
UNIQUE Foreign Key	Most common
Shared Primary Key	Stronger relationship

Fetch Data Using JOIN

INNER JOIN

SQL

```
SELECT users.name, user_profile.bio  
FROM users
```

```
INNER JOIN user_profile
ON users.id = user_profile.user_id;
```

Output

```
+-----+-----+
| name  | bio      |
+-----+-----+
| Aamir | Web Developer |
| Ali   | Designer   |
+-----+-----+
```

DELETE Behavior

Suppose:

SQL

```
DELETE FROM users WHERE id = 1;
```

What happens to profile?

Depends on:

SQL

```
ON DELETE
```

ON DELETE CASCADE

Automatically deletes child row.

SQL

```
FOREIGN KEY (user_id)
REFERENCES users(id)
ON DELETE CASCADE
```

Example

Delete user:

SQL

```
DELETE FROM users WHERE id = 1;
```

Then profile also deletes automatically.

ON UPDATE CASCADE

If parent ID changes, child updates automatically.

SQL

```
ON UPDATE CASCADE
```

Full Example with Cascade

SQL

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);  
  
CREATE TABLE profiles (  
    id INT PRIMARY KEY,  
    user_id INT UNIQUE,  
    bio VARCHAR(100),  
  
    FOREIGN KEY (user_id)  
    REFERENCES users(id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```


Advantages of One-to-One Relationship

Advantage	Explanation
Better organization	Separate sensitive data
Security	Store private data separately
Flexibility	Optional information
Faster queries	Smaller tables

When to Use One-to-One

Use when:

- ✔ Data is optional
- ✔ Data is sensitive
- ✔ Large data should be separated
- ✔ Logical separation needed

Example: User + Address

SQL	
users	
id	name
1	Aamir

SQL	
address	

id	user_id	city
1	1	Delhi

One user → One address

One-to-One vs One-to-Many

Feature	One-to-One	One-to-Many
Child rows	Only 1	Multiple
UNIQUE needed?	Yes	No
Example	User → Profile	User → Orders

One-to-One vs Many-to-Many

Relationship	Example
One-to-One	Person → Passport
One-to-Many	Customer → Orders
Many-to-Many	Students ↔ Courses

Common Interview Questions

Q1: How do you create One-to-One relationship?

Using:

- Foreign Key
- UNIQUE constraint

Q2: Why UNIQUE is required?

To prevent multiple child rows for same parent.

Q3: Difference between One-to-One and One-to-Many?

One-to-One allows only one related row.

One-to-Many allows many related rows.

Best Practice

Most developers use:

SQL

```
FOREIGN KEY + UNIQUE
```

because it is flexible and easy.

Final Complete Example

SQL

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  name VARCHAR(50)  
);  
  
CREATE TABLE profiles (  
  id INT PRIMARY KEY,  
  user_id INT UNIQUE,  
  bio VARCHAR(100),  
  
  FOREIGN KEY (user_id)  
  REFERENCES users(id)  
  ON DELETE CASCADE  
);
```

Summary

One-to-One Relationship

One row ↔ One row

Created using:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE

Short Formula

Foreign Key + UNIQUE = One-to-One



One-to-Few Relationship Using `populate()` in MongoDB + Mongoose

In MongoDB/Mongoose, a **One-to-Few** relationship means:

One document → Few related documents

Usually implemented using:

- `ObjectId`
- `ref`
- `populate()`

Real Example

User → Few Phone Numbers

One user can have:

- Personal number
- Office number
- Emergency number

Only a few phone numbers.

How populate() Works

populate() replaces referenced IDs with actual document data.

Step 1: Create Phone Model

phone.js

JavaScript

```
const mongoose = require("mongoose");

const phoneSchema = new mongoose.Schema({
  number: String
});

const Phone = mongoose.model("Phone", phoneSchema);

module.exports = Phone;
```

Step 2: Create User Model

user.js

JavaScript

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({

  name: String,

  phones: [
```

```
    {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Phone"
    }
  ]

});

const User = mongoose.model("User", userSchema);

module.exports = User;
```

Important Part

JavaScript

```
phones: [
  {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Phone"
  }
]
```

This means:

One User → Few Phone Documents

Step 3: Insert Data

Create Phone Documents

JavaScript

```
const Phone = require("./models/phone");

const p1 = await Phone.create({
  number: "9999999999"
});

const p2 = await Phone.create({
```

```
    number: "8888888888"  
  });
```

Step 4: Create User

JavaScript

```
const User = require("./models/user");  
  
const user = await User.create({  
  name: "Aamir",  
  phones: [p1._id, p2._id]  
});
```

Database Storage

User Document

JSON

```
{  
  "_id": "u1",  
  "name": "Aamir",  
  "phones": ["p1", "p2"]  
}
```

Only IDs are stored.

Step 5: Use populate()

JavaScript

```
const data = await User.findOne({ name: "Aamir" })  
  .populate("phones");  
  
console.log(data);
```

Output

JSON

```
{
  "_id": "u1",
  "name": "Aamir",

  "phones": [
    {
      "_id": "p1",
      "number": "9999999999"
    },
    {
      "_id": "p2",
      "number": "8888888888"
    }
  ]
}
```

What populate() Did

Before populate:

JSON

```
"phones": ["p1", "p2"]
```

After populate:

JSON

```
"phones": [
  { full phone document },
  { full phone document }
]
```

Visual Flow



One-to-Few Structure

One User → Few Phones

This is implemented using:

Array of ObjectIds

Limiting to Few Records

Suppose:

Maximum 3 phone numbers

Before adding:

JavaScript

```

if(user.phones.length >= 3){
  throw new Error("Maximum 3 phones allowed");
}

```

Example

Add New Phone

JavaScript

```
const newPhone = await Phone.create({
  number: "7777777777"
});

user.phones.push(newPhone._id);

await user.save();
```

Prevent 4th Phone

JavaScript

```
if(user.phones.length >= 3){
  console.log("Limit reached");
}
```

Complete Example

User Schema

JavaScript

```
const userSchema = new mongoose.Schema({

  name: String,

  phones: [
    {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Phone"
    }
  ]
});
```

Populate Query

JavaScript


```
const user = await User.findById(id)
    .populate("phones");
```

Why Use populate() ?

Benefit	Explanation
Cleaner database	Store references only
Faster updates	Separate documents
Reusable data	Same document reusable
Easy JOIN-like feature	Similar to SQL JOIN

populate() vs SQL JOIN

SQL	MongoDB
JOIN	populate()
Foreign Key	ObjectId
Table Relation	Document Reference

SQL Equivalent

```
SQL

SELECT *
FROM users
JOIN phones
ON users.id = phones.user_id;
```

MongoDB equivalent:

JavaScript

```
.populate("phones")
```

Real-World Examples

Parent

Few Child Docs

User

Addresses

Product

Images

Student

Emergency Contacts

Profile

Social Links

Important Interview Point

If interviewer asks:

“How do you implement One-to-Few in Mongoose?”

Answer:

Use an array of `ObjectId` references in the parent document and fetch related documents using `populate()`.

Final Summary

One-to-Few in Mongoose

One Document → Few Related Documents

Implemented using:

- `ObjectId`
- `ref`

- `populate()`

Formula

Array of ObjectIds + `populate()`

Main Syntax

Reference

JavaScript

```
type: mongoose.Schema.Types.ObjectId
```

Populate

JavaScript

```
.populate("fieldName")
```



one to squillions (in Details)

One-to-Squillions Relationship (Detailed Explanation)

What is One-to-Squillions?

“One-to-Squillions” is a database design term mostly used in:

- MongoDB
- NoSQL systems

It means:

One parent document → Massive number of child documents

The child records can become:

- Thousands
- Millions
- Billions

Meaning of “Squillions”

“Squillions” is not a real number.

It informally means:

Very huge amount

Real-World Examples

Parent	Massive Child Records
User	Logs
Post	Comments
Video	Views
Website	Analytics events
IoT Device	Sensor readings
Customer	Transactions

Example

Video → Views

One video can have:

Millions of views

Wrong Design (Embedding)

Suppose you store all views inside video document.

JSON

```
{
  "title": "MongoDB Tutorial",

  "views": [
    {},
    {},
    {},
    ...
    millions of records
  ]
}
```

Problem

This becomes VERY BAD because:

- ✗ Huge document size
- ✗ Slow queries
- ✗ Memory issues
- ✗ Update problems
- ✗ MongoDB document size limit (16MB)

Correct Design

Store child documents separately.

Structure

One Parent → Separate Massive Child Collection

Example Collections

videos Collection

JSON

```
{
  "_id": "v1",
  "title": "MongoDB Tutorial"
}
```

views Collection

JSON

```
{
  "_id": "view1",
  "video_id": "v1",
  "user": "Aamir",
  "time": "2026-05-08"
}
```

Relationship

One Video → Millions of Views

Why Separate Collection?

Because child records grow infinitely.

One-to-Squillions in MongoDB

MongoDB best practice:

- ✓ Parent document stays small
- ✓ Child documents stored separately
- ✓ Use references

Mongoose Implementation

Step 1: Create Parent Model

video.js

JavaScript

```
const mongoose = require("mongoose");

const videoSchema = new mongoose.Schema({
  title: String
});

module.exports = mongoose.model("Video", videoSchema);
```

Step 2: Create Child Model

view.js

JavaScript

```
const mongoose = require("mongoose");

const viewSchema = new mongoose.Schema({

  video: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Video"
  },
```

```
    user: String,

    viewedAt: {
      type: Date,
      default: Date.now
    }
  });

module.exports = mongoose.model("View", viewSchema);
```

Important Design

Notice:

JavaScript

```
video: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "Video"
}
```

This creates:

Many child documents referencing one parent

Insert Example

Create Video

JavaScript

```
const video = await Video.create({
  title: "MongoDB Tutorial"
});
```

Add Millions of Views

JavaScript

```
await View.create({  
  video: video._id,  
  user: "Aamir"  
});
```

Each view becomes separate document.

Why This Is Best?

Because:

Views can grow infinitely

without affecting parent document size.

Query Example

Find Views of Video

JavaScript

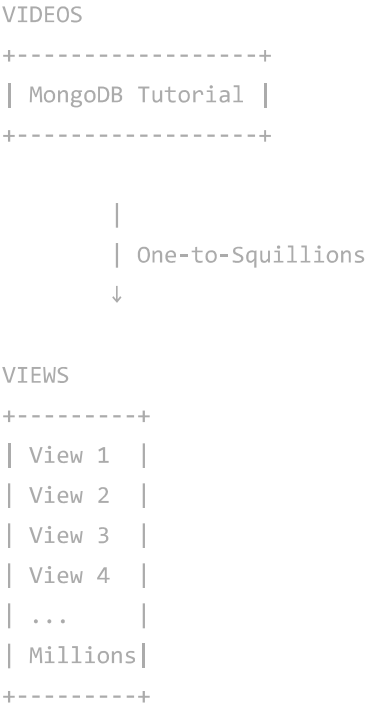
```
const views = await View.find({  
  video: videoId  
});
```

Count Total Views

JavaScript

```
const total = await View.countDocuments({  
  video: videoId  
});
```

Visual Diagram



Difference from One-to-Few

Feature	One-to-Few	One-to-Squillions
Child count	Small	Massive
Storage	Array possible	Separate collection
Embedding	Good	Bad
Performance	Easy	Needs optimization

Embedding vs Referencing

One-to-Few

Embedding often works:

JSON

```
{
  "phones": [
    "9999999999",
    "8888888888"
  ]
}
```

One-to-Squillions

Embedding becomes dangerous:

JSON

```
{
  "comments": [
    millions...
  ]
}
```

✗ Very bad design.

Best Practice

For One-to-Squillions:

- ✓ Use separate collection
- ✓ Use references
- ✓ Add indexes
- ✓ Use pagination
- ✓ Use aggregation

Indexing Important

Because millions of records exist.

Add index:

JavaScript

```
viewSchema.index({ video: 1 });
```

Makes queries fast.

Pagination Example

Never load millions at once.

Use:

JavaScript

```
const comments = await Comment.find({
  post: postId
})
.limit(20)
.skip(40);
```

Common Use Cases

Parent	Squillions Child Docs
Post	Comments
User	Activity Logs
Product	Reviews
Server	Logs
Device	Sensor Data

SQL Equivalent

In SQL:

One-to-Squillions behaves like One-to-Many

but with:

- Huge scaling
- Performance optimization
- Partitioning
- Indexing

Important MongoDB Principle

MongoDB rule:

Data that grows without bound should NOT be embedded

Very important interview point.

Interview Question

Q1: What is One-to-Squillions?

Answer:

A relationship where one parent has an extremely large number of child documents.

Q2: Why avoid embedding?

Because:

- Huge document size
- Performance issues
- MongoDB 16MB limit

Q3: Best implementation?

Use:

Separate child collection with references

Final Summary

One-to-Squillions

One Parent → Massive Number of Child Documents

Best Design



Formula

One-to-Squillions = Scalable One-to-Many

Main Rule

Never embed infinitely growing data



Reading Docs

Documentation ("Docs") is the official guide that explains:

- How a technology works
- Installation steps
- Syntax
- Methods/functions
- Examples
- Best practices

Learning to read docs is one of the most important developer skills.

What Are Docs?

Docs are like a:

User Manual for Developers

Examples:

- React Docs
- MongoDB Docs
- Node.js Docs
- Express Docs
- Mongoose Docs

Why Reading Docs Is Important

Most professional developers learn by:

- ✓ Reading docs
- ✓ Testing examples
- ✓ Experimenting

Not by memorizing tutorials.

Benefits of Reading Docs

Benefit	Explanation
Latest Information	Docs stay updated
Accurate Syntax	Official source
Better Understanding	Learn concepts deeply
Debugging Help	Find solutions quickly
Interview Preparation	Understand real concepts

Example Documentation Websites

React

[React Docs](#)

MongoDB

[MongoDB Docs](#)

Mongoose

[Mongoose Docs](#)

Node.js

[Node.js Docs](#)

Express.js

[Express Docs](#)

How to Read Docs Properly

Most beginners make mistake:

- ✗ Reading every line
- ✗ Memorizing syntax only

Correct method:

- ✓ Learn structure
- ✓ Understand examples
- ✓ Practice immediately

Step-by-Step Method

Step 1: Understand the Purpose

Ask:

```
What does this library/framework do?
```

Example:

- Express → Backend framework
- Mongoose → MongoDB ODM
- React → UI library

Step 2: Read Installation First

Almost every docs page starts with installation.

Example:

Bash

```
npm install mongoose
```

This tells you:

- package name
- how to setup

Step 3: Read Basic Example

Docs usually provide a simple example.

Example from Mongoose:

JavaScript

```
const mongoose = require("mongoose");

mongoose.connect("mongodb://127.0.0.1/test");
```

Do NOT just copy.

Understand:

- What connect() does
- Why URL is needed
- What database name means

Step 4: Understand Main Concepts

Example:

In Mongoose

Main concepts are:

- Schema
- Model
- Document
- Validation
- Populate

Step 5: Read Syntax Carefully

Example:

JavaScript

```
User.find({})
```

Ask:

```
What is find()?
What does {} mean?
What does it return?
```

Step 6: Test Everything Yourself

Reading alone is not enough.

Always:

```
Read → Practice → Experiment
```

Example

After reading:

JavaScript

```
User.find({})
```

Try:

JavaScript

```
User.find({ age: 18 })
```

Experimenting improves understanding.

How Professional Developers Read Docs

Professionals do NOT read full docs.

They:

- ✔ Search exact problem
- ✔ Read examples
- ✔ Check syntax
- ✔ Test quickly

Smart Way to Read Docs

Suppose you need:

```
populate() in Mongoose
```

Search:

```
mongoose populate docs
```

Read only relevant section.

Important Parts of Documentation

Section	Purpose
Introduction	What tool does
Installation	Setup
Quick Start	Basic usage
API Reference	Methods/functions
Examples	Real code
Guides/Tutorials	Detailed explanation

Understanding API Reference

Example:

JavaScript

```
Model.find()
```

Docs explain:

- Parameters
- Return type
- Examples
- Options

Example Breakdown

JavaScript

```
User.find({ age: 18 })
```

Part	Meaning
User	Model
find()	Query method
{ age: 18 }	Filter

Reading Errors in Docs

Docs also explain:

- Errors
- Warnings
- Edge cases

Very important for debugging.

Common Beginner Mistakes

Mistake 1

Trying to memorize everything.

✗ Wrong

Mistake 2

Skipping examples.

✗ Wrong

Mistake 3

Copy-paste without understanding.

✗ Wrong

Mistake 4

Fear of large documentation.

✗ Wrong

Even seniors search docs daily.

Best Strategy

Learn in Layers

First Read

Understand overview.

Second Read

Understand syntax.

Third Read

Build project using it.

Example: Reading Mongoose Docs

Learn Order

1. Installation

Bash

```
npm install mongoose
```

2. Connection

JavaScript

```
mongoose.connect()
```

3. Schema

JavaScript

```
new mongoose.Schema()
```

4. Model

JavaScript

```
mongoose.model()
```

5. CRUD

JavaScript

```
find()  
save()  
updateOne()  
deleteOne()
```

6. Populate

JavaScript

```
populate()
```

How to Practice Reading Docs

Method

1. Pick one topic
2. Read docs for 20 mins
3. Write code yourself
4. Break code intentionally
5. Fix using docs

This builds real skill.

Golden Rule

```
Documentation is not for memorizing.  
It is for understanding and referencing.
```

Reading Docs Efficiently

Use:

Ctrl + F

Search keywords quickly.

Example:

```
populate
middleware
validation
```

Use Official Docs Only

Prefer official documentation over random blogs.

Because:

- ✓ Updated
- ✓ Accurate
- ✓ Trusted

Docs vs Tutorials

Tutorials

Step-by-step

Beginner friendly

Slower learning

Guided

Docs

Reference-based

Detailed

Faster growth

Self-learning

Best approach:

Tutorial + Docs together

Real Developer Workflow

Problem →
Search Docs →
Read Example →
Test →
Fix →
Repeat

Interview Tip

If interviewer asks:

“Do developers memorize everything?”

Best answer:

No. Professional developers regularly use documentation to understand APIs, syntax, and best practices.

Final Summary

Documentation Helps You

- ✓ Learn technology
- ✓ Understand syntax
- ✓ Solve errors
- ✓ Build projects
- ✓ Become independent developer

Best Formula

Read → Understand → Practice → Experiment

Most Important Skill

Developers who can read docs well learn faster than others.

